

GitOps the Hard Way

Rebuilding a Source of Truth for an EVPN Fabric
(Three Times)

Radim Roška · ALTEPRO solutions

AUTOCON S

THE NETWORK AUTOMATION CONFERENCE

CRA DC Migration



BEFORE:

- Legacy DC services
- Unicast + multicast variants
- No service catalog
- Knowledge in people and configs

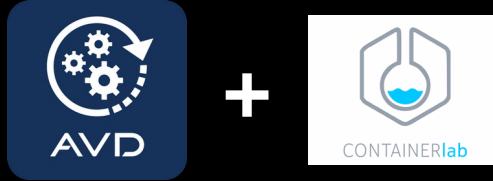
AFTER:

- EVPN/VXLAN fabric
- Explicit service model
- Fabric & Service SOT
- CI driven changes

· 2 DCs · ~40 ARISTA switches · automated from day one

False Confidence

- Dual-DC fabric in Containerlab:
running after a few days
 - ARISTA AVD – ready to use YAML data model



“Maybe AVD will do all the work for us?”

Before Migration: Define the Service Model
There was no catalog to import. We built one with CRA.

- 1. Define service catalog
 - types / variables / model schema
- 2. Fill migration workbench
 - existing services → structured rows in `xlsx`
- 3. Design `migrator.py`
 - `xlsx` → CRA service YAML (service SOT)

The Model Fought Back

- service identity = global
- VLAN/VNI mapping = POD-local

→ CRA SOT abstraction
 → translator.py
 → AVD data model

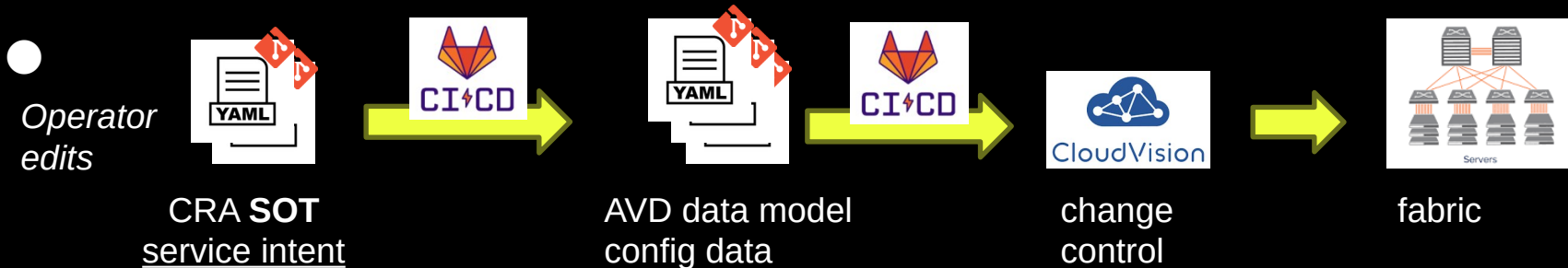
```
1  tenants:
2    - name: CRA
3      ...
4    vrfs:
5      - name: cra-inet
6        ...
7      svis:
8        - CRA_id: xx0086
9          name: EXAMPLE_SERVICE
10         CRA_service_type: service_l2
11         vxlan: true
12         enabled: true
13         CRA_pods:
14           DC1_POD1:
15             CRA_vlan-id: 86
16             CRA_nodes: [dca-xxx-dc-sw01,
17                        dca-xxx-dc-sw02]
18           DC1_POD2:
19             CRA_vlan-id: 186
20             CRA_nodes: [SWITCH_GROUP1]
```

From Service Model to Config: Two Phases

● Phase 1: Migration *(make migration possible)*



Phase 2: Day 2 Operations *(make operations safe)*



CI pipeline: validate · translate · build · deploy
Day 2 tools: avd-editor · avd-validator · avd-finder

One Year Later

- The editor helped. The validator mattered.
- Operators run it daily
 - guided editor or YAML
 - Git/MR workflow
 - avd-finder became essential
- Cleaner data + mindset change

```
- CRA_id: 9999
  name: TEST_DEMO
  CRA_service_type: service_l2
  CRA_pods:
    TVPM_DCI:
      CRA_vlan-id: 999
      CRA_nodes:
        - dca-phdc-bl01
        - dca-p6dc-bl02
  enabled: true
  vxlan: true
```

No associated IPv4 static routes found.

No associated IPv6 static routes found.

Are you satisfied with this service? [y/n] (y):

Services YAML file updated successfully, preserving or

Do you want to commit this service? [y/n] (y):

Enter commit message (Create service_l2 TEST_DEMO(9999

Changes committed successfully.

Do you want to make more changes? [y/n] (n):

Changes pushed successfully.

Create a merge request using this link: [Click here](#)

Or use the full URL below if your terminal doesn't sup

<https://gitlab-test.altepro.cz/root/test-dccore/-/merge>

[%5D-529&merge_request%5Btarget_branch%5D=main](#)

Deleted feature branch: TEST_ADD_SERVICE_SETINAV

edit → commit → MR → validate → translate → build → deploy

What We Learned

Not three rewrites → Three levels of understanding.

- Fabric is easy → legacy services are hard to model
- Service intent needs its own abstraction
- SOT → validation → workflow

We didn't just hand over a fabric.

We built a way to run it safely – every day



Questions?

CI Workflow (backup slide)

